

```
In [3]: import os
        print(os.getcwd())
```

C:\Users\sande

```
In [ ]: df = pd.read_csv(file_path)
```

```
In [12]: import pandas as pd
        import numpy as np
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import mean_absolute_error, mean_squared_error
        import xgboost as xgb

        # Load your dataset
        df = pd.read_csv("cltv3.csv")

        # Feature Engineering
        df_features = pd.DataFrame()
        df_features["Frequency"] = df["Total_Transactions"]
        df_features["Recency"] = df["Last_Transaction_Days_Ago"]
        df_features["AOV"] = df["Total_Spent"] / df["Total_Transactions"]
        df_features["LTV"] = df["LTV"]

        # Define features and target
        X = df_features[["Frequency", "Recency", "AOV"]]
        y = df_features["LTV"]

        # Train/test split
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

        # Train XGBoost model
        model = xgb.XGBRegressor(objective="reg:squarederror", random_state=42)
        model.fit(X_train, y_train)

        # Predict and evaluate
        y_pred = model.predict(X_test)
        mae = mean_absolute_error(y_test, y_pred)
        rmse = np.sqrt(mean_squared_error(y_test, y_pred))

        print("Mean Absolute Error (MAE):", mae)
        print("Root Mean Squared Error (RMSE):", rmse)
```

Mean Absolute Error (MAE): 8281.751643943688

Root Mean Squared Error (RMSE): 10779.536016255563

```
In [14]: df["Frequency"] = df["Total_Transactions"]
```

```
In [16]: df["Recency"] = df["Last_Transaction_Days_Ago"]
```

```
In [20]: df["AOV"] = df["Total_Spent"] / df["Total_Transactions"]
```

```
In [22]: df_features = pd.DataFrame()
        df_features["Frequency"] = df["Total_Transactions"]
```

```
df_features["Recency"] = df["Last_Transaction_Days_Ago"]
df_features["AOV"] = df["Total_Spent"] / df["Total_Transactions"]
```

```
In [26]: print(df_features[["Frequency", "Recency", "AOV"]]) # Print all rows
```

	Frequency	Recency	AOV
0	192	209	16736.384023
1	979	240	14536.734683
2	329	21	7061.372800
3	71	285	16426.876453
4	878	329	10800.092660
...	...	...	...
6995	340	20	162.703503
6996	647	252	1575.707877
6997	481	348	14429.355178
6998	574	50	8005.027184
6999	825	189	15697.668867

[7000 rows x 3 columns]

```
In [28]: pip install xgboost
```

Requirement already satisfied: xgboost in c:\users\sande\anaconda3\lib\site-packages (3.0.0)
 Requirement already satisfied: numpy in c:\users\sande\anaconda3\lib\site-packages (from xgboost) (1.26.4)
 Requirement already satisfied: scipy in c:\users\sande\anaconda3\lib\site-packages (from xgboost) (1.13.1)
 Note: you may need to restart the kernel to use updated packages.

```
In [50]: df = pd.read_csv('C:\Users\sande\Downloads\cltv3') # Forward slashes
```

Cell In[50], line 1

```
df = pd.read_csv('C:\Users\sande\Downloads\cltv3') # Forward slashes
```

SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 3-4: truncated \UXXXXXXXX escape

```
In [52]: import pandas as pd
```

```
# Ensure the correct file path and format
file_path = r'C:\Users\sande\Downloads\cltv3.csv'

# Attempt to read the CSV file
try:
    df = pd.read_csv(file_path)
    print("File loaded successfully!")
except FileNotFoundError as e:
    print(f"Error: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

Error: [Errno 2] No such file or directory: 'C:\\Users\\sande\\Downloads\\cltv3.csv'

```
In [56]: df = pd.read_csv(r'C:\Users\sande\Downloads\cltv3new.csv')
```

```

-----
OSError                                Traceback (most recent call last)
Cell In[56], line 1
----> 1 df = pd.read_csv(r"C:\Users\sande\Downloads\cltv3new.csv")

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision, storage_options, dtype_backend)
    1013 kwds_defaults = _refine_defaults_read(
    1014     dialect,
    1015     delimiter,
    (...)
    1022     dtype_backend=dtype_backend,
    1023 )
    1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:620, in _read(filepath_or_buffer, kwds)
    617 _validate_names(kwds.get("names", None))
    619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
    622 if chunksize or iterator:
    623     return parser

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1620, in TextFileReader.__init__(self, f, engine, **kwds)
    1617 self.options["has_index_names"] = kwds["has_index_names"]
    1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)

File ~\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1880, in TextFileReader._make_engine(self, f, engine)
    1878 if "b" not in mode:
    1879     mode += "b"
-> 1880 self.handles = get_handle(
    1881     f,
    1882     mode,
    1883     encoding=self.options.get("encoding", None),
    1884     compression=self.options.get("compression", None),
    1885     memory_map=self.options.get("memory_map", False),
    1886     is_text=is_text,
    1887     errors=self.options.get("encoding_errors", "strict"),
    1888     storage_options=self.options.get("storage_options", None),
    1889 )
    1890 assert self.handles is not None
    1891 f = self.handles.handle

File ~\anaconda3\Lib\site-packages\pandas\io\common.py:873, in get_handle(path_or_buffer, mode, encoding, compression, memory_map, is_text, errors, storage_options)

```

```

868 elif isinstance(handle, str):
869     # Check whether the filename is to be opened in binary mode.
870     # Binary mode does not support 'encoding' and 'newline'.
871     if ioargs.encoding and "b" not in ioargs.mode:
872         # Encoding
--> 873         handle = open(
874             handle,
875             ioargs.mode,
876             encoding=ioargs.encoding,
877             errors=errors,
878             newline="",
879         )
880     else:
881         # Binary mode
882         handle = open(handle, ioargs.mode)

```

OSError: [Errno 22] Invalid argument: '"C:\\Users\\sande\\Downloads\\cltv3new.csv"'

```
In [58]: df = pd.read_csv(r'C:\Users\sande\Downloads\cltv3new.csv')
```

```
In [60]: import pandas as pd
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load the data
df = pd.read_csv(r'C:\Users\sande\Downloads\cltv3new.csv') # Update path if needed

# Check the data
print(df.head())
print(df.columns)

# 1. Define features (X) and target (y)
# Replace 'target_column_name' with the actual column name you're predicting
X = df.drop(columns='target_column_name') # features
y = df['target_column_name']             # target

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Initialize and train the model
model = xgb.XGBRegressor(objective='reg:squarederror', random_state=42)
model.fit(X_train, y_train)

# 4. Make predictions
y_pred = model.predict(X_test)

# 5. Evaluate the model
rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.2f}")

```

	Customer_ID	Age	Location	Income_Level	Total_Transactions	\
0	cust_0000	54	Urban	Low	192	
1	cust_0001	67	Suburban	High	979	
2	cust_0002	44	Urban	High	329	
3	cust_0003	30	Rural	High	71	
4	cust_0004	58	Urban	Middle	878	

	Avg_Transaction_Value	Max_Transaction_Value	Min_Transaction_Value	\
0	16736.384023	60216.834510	6525.814861	
1	14536.734683	48350.100272	2186.742245	
2	7061.372800	32521.157187	2743.406808	
3	16426.876453	17827.896720	4360.784994	
4	10800.092660	17497.634534	4532.872520	

	Total_Spent	Active_Days	Last_Transaction_Days_Ago	\
0	3.213386e+06	140	209	
1	1.423146e+07	229	240	
2	2.323192e+06	73	21	
3	1.166308e+06	299	285	
4	9.482481e+06	236	329	

	Loyalty_Points_Earned	Referral_Count	Cashback_Received	\
0	2114	25	2224.012140	
1	2960	20	4026.823518	
2	3170	0	1441.011395	
3	4756	35	4365.855580	
4	1992	18	4161.523827	

	App_Usage_Frequency	Preferred_Payment_Method	Support_Tickets_Raised	\
0	Monthly	Debit Card	3	
1	Monthly	UPI	17	
2	Monthly	Debit Card	11	
3	Weekly	Wallet Balance	6	
4	Daily	UPI	18	

	Issue_Resolution_Time	Customer_Satisfaction_Score	LTV
0	61.568590	1	3.279546e+05
1	60.392889	8	1.437053e+06
2	45.305579	4	2.419387e+05
3	22.030191	1	1.284599e+05
4	20.634723	5	9.569514e+05

```
Index(['Customer_ID', 'Age', 'Location', 'Income_Level', 'Total_Transactions',
      'Avg_Transaction_Value', 'Max_Transaction_Value',
      'Min_Transaction_Value', 'Total_Spent', 'Active_Days',
      'Last_Transaction_Days_Ago', 'Loyalty_Points_Earned', 'Referral_Count',
      'Cashback_Received', 'App_Usage_Frequency', 'Preferred_Payment_Method',
      'Support_Tickets_Raised', 'Issue_Resolution_Time',
      'Customer_Satisfaction_Score', 'LTV'],
      dtype='object')
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[60], line 15
     11 print(df.columns)
     13 # 1. Define features (X) and target (y)
     14 # Replace 'target_column_name' with the actual column name you're predicting
--> 15 X = df.drop(columns='target_column_name') # features
     16 y = df['target_column_name']           # target
     18 # 2. Train-test split

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:5581, in DataFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
    5433 def drop(
    5434     self,
    5435     labels: IndexLabel | None = None,
    (...)
    5442     errors: IgnoreRaise = "raise",
    5443 ) -> DataFrame | None:
    5444     """
    5445     Drop specified labels from rows or columns.
    5446
    (...)
    5579         weight 1.0      0.8
    5580     """
-> 5581     return super().drop(
    5582         labels=labels,
    5583         axis=axis,
    5584         index=index,
    5585         columns=columns,
    5586         level=level,
    5587         inplace=inplace,
    5588         errors=errors,
    5589     )

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4788, in NDFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
    4786 for axis, labels in axes.items():
    4787     if labels is not None:
-> 4788         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
    4790 if inplace:
    4791     self._update_inplace(obj)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4830, in NDFrame._drop_axis(self, labels, axis, level, errors, only_slice)
    4828     new_axis = axis.drop(labels, level=level, errors=errors)
    4829     else:
-> 4830     new_axis = axis.drop(labels, errors=errors)
    4831     indexer = axis.get_indexer(new_axis)
    4833 # Case for non-unique axis
    4834 else:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:7070, in Index.drop(self, labels, errors)
    7068 if mask.any():
    7069     if errors != "ignore":
-> 7070         raise KeyError(f"{labels[mask].tolist()} not found in axis")

```

```

7071     indexer = indexer[~mask]
7072     return self.delete(indexer)

```

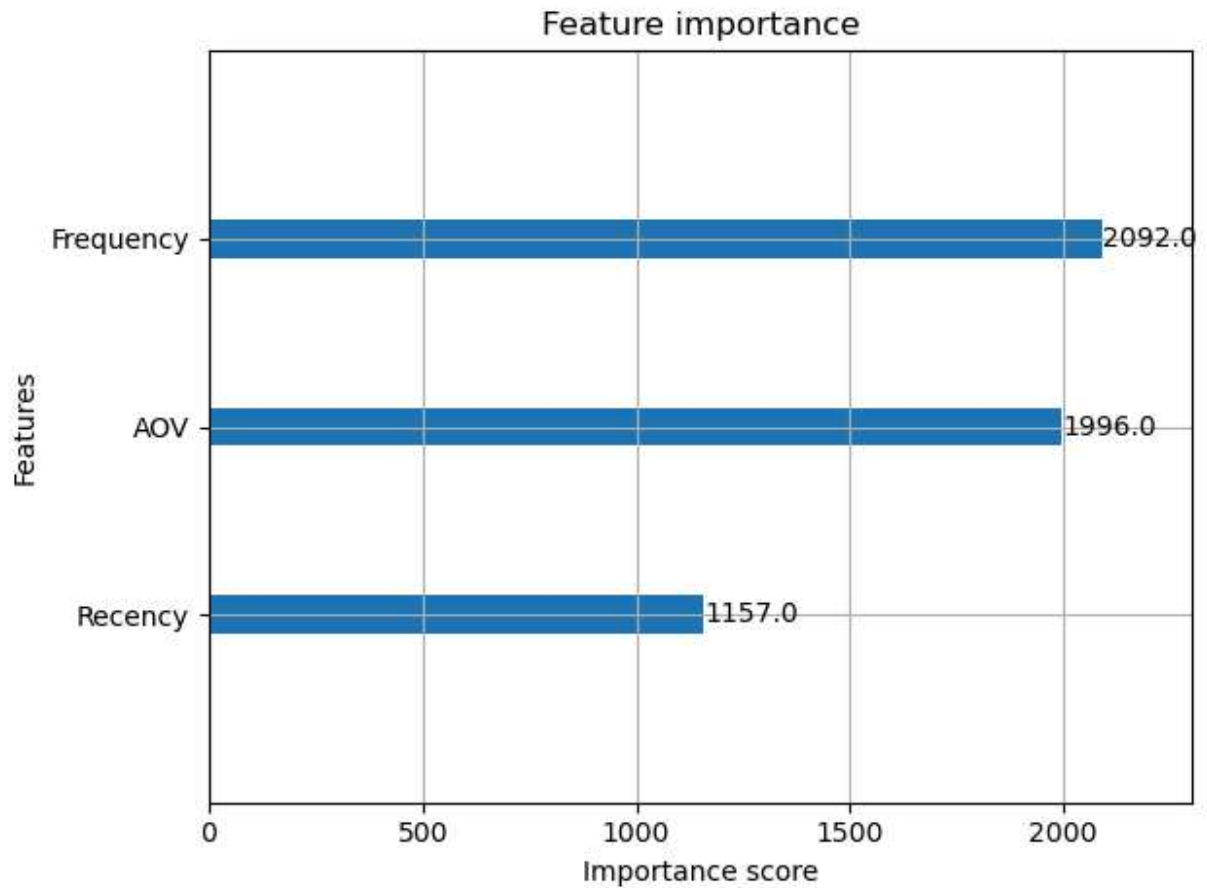
KeyError: "['target_column_name'] not found in axis"

```
In [62]: import matplotlib.pyplot as plt
```

```

xgb.plot_importance(model)
plt.tight_layout()
plt.show()

```



```
In [64]: df['predicted_ltv'] = model.predict(X)
```

```
In [66]: df['ltv_segment'] = pd.qcut(df['predicted_ltv'], q=3, labels=['Low', 'Medium', 'High'])
```

```

In [72]: segment_summary = df.groupby('ltv_segment').agg({
        'predicted_ltv': ['mean', 'count'],
        'total_orders': 'mean',
        'avg_order_value': 'mean',
        'tenure_months': 'mean'
    }).round(2)

print(segment_summary)

```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[72], line 1
----> 1 segment_summary = df.groupby('ltv_segment').agg({
      2     'predicted_ltv': ['mean', 'count'],
      3     'total_orders': 'mean',
      4     'avg_order_value': 'mean',
      5     'tenure_months': 'mean'
      6 }).round(2)
      7
      8 print(segment_summary)

File ~\anaconda3\Lib\site-packages\pandas\core\groupby\generic.py:1432, in DataFrameGroupBy.aggregate(self, func, engine, engine_kwargs, *args, **kwargs)
    1429     kwargs["engine_kwargs"] = engine_kwargs
    1431 op = GroupByApply(self, func, args=args, kwargs=kwargs)
-> 1432 result = op.agg()
    1433 if not is_dict_like(func) and result is not None:
    1434     # GH #52849
    1435     if not self.as_index and is_list_like(func):

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:190, in Apply.agg(self)
    187     return self.apply_str()
    189 if is_dict_like(func):
--> 190     return self.agg_dict_like()
    191 elif is_list_like(func):
    192     # we require a list, but not a 'str'
    193     return self.agg_list_like()

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:423, in Apply.agg_dict_like(self)
    415 def agg_dict_like(self) -> DataFrame | Series:
    416     """
    417     Compute aggregation in the case of a dict-like argument.
    418     (...)
    421     Result of aggregation.
    422     """
--> 423     return self.agg_or_apply_dict_like(op_name="agg")

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:1608, in GroupByApply.agg_or_apply_dict_like(self, op_name)
    1603     kwargs.update({"engine": engine, "engine_kwargs": engine_kwargs})
    1605 with com.temp_setattr(
    1606     obj, "as_index", True, condition=hasattr(obj, "as_index")
    1607 ):
-> 1608     result_index, result_data = self.compute_dict_like(
    1609         op_name, selected_obj, selection, kwargs
    1610     )
    1611 result = self.wrap_results_dict_like(selected_obj, result_index, result_data)
    1612 return result

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:462, in Apply.compute_dict_like(self, op_name, selected_obj, selection, kwargs)
    460 is_groupby = isinstance(obj, (DataFrameGroupBy, SeriesGroupBy))
    461 func = cast(AggFuncTypeDict, self.func)

```

```
--> 462 func = self.normalize_dictlike_arg(op_name, selected_obj, func)
464 is_non_unique_col = (
465     selected_obj.ndim == 2
466     and selected_obj.columns.nunique() < len(selected_obj.columns)
467 )
469 if selected_obj.ndim == 1:
470     # key only used for output
```

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:663, in Apply.normalize_dictlike_arg(self, how, obj, func)

```
661     cols = Index(list(func.keys())).difference(obj.columns, sort=True)
662     if len(cols) > 0:
--> 663         raise KeyError(f"Column(s) {list(cols)} do not exist")
665 aggregator_types = (list, tuple, dict)
667 # if we have a dict of any non-scalars
668 # eg. {'A' : ['mean']}, normalize all to
669 # be list-likes
670 # Cannot use func.values() because arg may be a Series
```

KeyError: "Column(s) ['avg_order_value', 'tenure_months', 'total_orders'] do not exist"

In [74]: `print(df.columns.tolist())`

```
['Customer_ID', 'Age', 'Location', 'Income_Level', 'Total_Transactions', 'Avg_Transaction_Value', 'Max_Transaction_Value', 'Min_Transaction_Value', 'Total_Spent', 'Active_Days', 'Last_Transaction_Days_Ago', 'Loyalty_Points_Earned', 'Referral_Count', 'Cashback_Received', 'App_Usage_Frequency', 'Preferred_Payment_Method', 'Support_Tickets_Raised', 'Issue_Resolution_Time', 'Customer_Satisfaction_Score', 'LTV', 'predicted_ltv', 'ltv_segment']
```

In [78]: `segment_summary = df.groupby('ltv_segment').mean(numeric_only=True).round(2)`
`print(segment_summary)`

ltv_segment	Age	Total_Transactions	Avg_Transaction_Value	\
0	42.34	352.97	6905.20	
1	43.21	607.28	12118.64	
2	42.52	811.61	16323.40	

ltv_segment	Max_Transaction_Value	Min_Transaction_Value	Total_Spent	\
0	20760.46	2057.01	1714362.91	
1	36901.47	3623.12	6741720.34	
2	49802.11	4957.47	13127058.49	

ltv_segment	Active_Days	Last_Transaction_Days_Ago	Loyalty_Points_Earned	\
0	181.56	184.34	2500.30	
1	184.57	182.64	2514.73	
2	177.91	184.55	2479.20	

ltv_segment	Referral_Count	Cashback_Received	Support_Tickets_Raised	\
0	25.09	2507.96	10.14	
1	24.78	2502.33	9.96	
2	24.06	2444.25	9.68	

ltv_segment	Issue_Resolution_Time	Customer_Satisfaction_Score	LTV	\
0	36.53	5.47	183158.58	
1	36.55	5.47	685895.89	
2	36.48	5.55	1324393.67	

ltv_segment	predicted_ltv
0	1.832443e+05
1	6.859312e+05
2	1.324791e+06

```
In [80]: # Make sure your DataFrame has these columns: customer_id, predicted_ltv, ltv_segme
final_df = df[['customer_id', 'predicted_ltv', 'ltv_segment']]

# Export to CSV
final_df.to_csv('final_ltv_predictions.csv', index=False)

print("✅ Final LTV predictions saved as 'final_ltv_predictions.csv'")
```

```

-----
KeyError                                Traceback (most recent call last)
Cell In[80], line 2
      1 # Make sure your DataFrame has these columns: customer_id, predicted_ltv, lt
v_segment
----> 2 final_df = df[['customer_id', 'predicted_ltv', 'ltv_segment']]
      4 # Export to CSV
      5 final_df.to_csv('final_ltv_predictions.csv', index=False)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:4108, in DataFrame.__getitem__(self, key)
    4106     if is_iterator(key):
    4107         key = list(key)
-> 4108     indexer = self.columns._get_indexer_strict(key, "columns")[1]
    4110 # take() does not accept boolean indexers
    4111 if getattr(indexer, "dtype", None) == bool:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6200, in Index._get_indexer_strict(self, key, axis_name)
    6197 else:
    6198     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
-> 6200 self._raise_if_missing(keyarr, indexer, axis_name)
    6202 keyarr = self.take(indexer)
    6203 if isinstance(key, Index):
    6204     # GH 42790 - Preserve name from an Index

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6252, in Index._raise_if_missing(self, key, indexer, axis_name)
    6249     raise KeyError(f"None of [{key}] are in the [{axis_name}]")
    6251 not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
-> 6252 raise KeyError(f"{not_found} not in index")

KeyError: "['customer_id'] not in index"

```

```
In [82]: print(df.columns)
```

```

Index(['Customer_ID', 'Age', 'Location', 'Income_Level', 'Total_Transactions',
      'Avg_Transaction_Value', 'Max_Transaction_Value',
      'Min_Transaction_Value', 'Total_Spent', 'Active_Days',
      'Last_Transaction_Days_Ago', 'Loyalty_Points_Earned', 'Referral_Count',
      'Cashback_Received', 'App_Usage_Frequency', 'Preferred_Payment_Method',
      'Support_Tickets_Raised', 'Issue_Resolution_Time',
      'Customer_Satisfaction_Score', 'LTV', 'predicted_ltv', 'ltv_segment'],
      dtype='object')

```

```
In [84]: # Make sure your DataFrame has the correct columns: 'Customer_ID', 'predicted_ltv',
final_df = df[['Customer_ID', 'predicted_ltv', 'ltv_segment']]
```

```

# Export to CSV
final_df.to_csv('final_ltv_predictions.csv', index=False)

print("✅ Final LTV predictions saved as 'final_ltv_predictions.csv'")

```

✅ Final LTV predictions saved as 'final_ltv_predictions.csv'

```
In [86]: # Load the saved CSV to verify
final_df_check = pd.read_csv('final_ltv_predictions.csv')
```

```
# Preview the first few rows  
print(final_df_check.head())
```

	Customer_ID	predicted_ltv	ltv_segment
0	cust_0000	329914.44	0
1	cust_0001	1435305.40	2
2	cust_0002	237631.36	0
3	cust_0003	130368.50	0
4	cust_0004	960119.40	1

In []: